

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf\_2a64bc88-2a8\agent-a81117999daa9a7f7.jsonl`
- SHA-256: `1ce309203124b4779ea44a433bfd4c82412a29a9f67d964395a48abbeff1ea50`
- Source modified: `2026-06-10T06:15:36+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf\_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T06:12:46.617Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("eng-pipeline") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"Result/Failure error contract is implemented only by the legacy 'motion' segmenter ? all four production segmenters still return [] on real failures","severity":"high","area":"segmenters / error contract","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/results.py:9-11","evidence":"results.py docstring: 'we start by applying it to the segmenter and provider layers (the biggest sources of silent [] returns today)'. Grep for Success(/Failure(across the repo: only backend/pipeline/segmenters/motion.py:33,226 and tests/test_base.py. wasb_hybrid.py returns bare [] for missing API key (:73), unknown provider (:78), invalid duration (:82), failed healthcheck (:109), and no trajectory (:118); tracknet_hybrid, yolo_hybrid, and gemini are identical. base.py:24 declares process_video -> Result[List[Dict]]; main.py:139/:399 checks isinstance(result, Failure) ? that branch can only fire for motion (which ironically is the segmenter with fabricated metadata).","why_it_matters":"The exact problem the Result type was built to fix ('empty != error') persists in every production path: a missing GEMINI_API_KEY or dead WSL env produces HTTP 200 {'status':'success', 'play_segments': []}. Only the post-hoc report rules (spec.yaml silent_provider_noop) disambiguate, and only if obsreport is installed. The async-job slice will need real terminal job states (failed vs empty) ? this contract gap lands directly in its path. Sharper than CODE_MAP's framing, which describes the [] behavior but not that the declared, partially-shipped contract is unimplemented where it matters."}

Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge

severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:12:54.576Z)

```
1      """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3      This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5      The goal is to make it impossible to confuse "no data / no rallies" with "something
6      actually failed"
7      (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
8
9      We use simple dataclasses so callers can use isinstance checks or structural pattern
10     matching.
11
12     This is intentionally small and incremental ? we start by applying it to the segmenter
13     and provider layers (the biggest sources of silent [] returns today).
14
15     The types are generic where it makes sense. Callers can still access .value on Success
16     or .message on Failure.
17
18     This module has no dependencies on the rest of the app.
19     """
20
21     from __future__ import annotations
22
23     from dataclasses import dataclass, field
24     from typing import Generic, TypeVar, List, Optional, Any, Union
25
26     T = TypeVar("T")
27
28     @dataclass
29     class Success(Generic[T]):
30         """Represents a successful outcome with an optional list of non-fatal warnings."""
31
32         value: T
33         warnings: List[str] = field(default_factory=list)
34
35         def is_success(self) -> bool:
36             return True
37
38         def unwrap(self) -> T:
39             """Return the value. Only safe to call if you already know it's a Success."""
40             return self.value
41
42     @dataclass
43     class Failure:
44         """Represents a failure with a human-readable message and optional details."""
45
46         message: str
47         details: Optional[dict[str, Any]] = None
48         is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
49
50         def is_success(self) -> bool:
51             return False
52
53         def unwrap(self) -> Any:
54             """Raise to make misuse obvious in code that assumes success."""
55             raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
56
57     # Convenient alias for use in type hints: Result[list[dict], Failure]
58     # Result[ValueType] means Success[ValueType] | Failure
```

```

59     # (the Failure "error" type is fixed for this contract)
60     Result = Union[Success[T], Failure]
61

```

3. user (2026-06-10T06:12:55.919Z)

```

backend\results.py:27:class Success(Generic[T]):
tests\test_base.py:17:         return Failure(message="Simulated failure for test",
is_recoverable=True)
tests\test_base.py:18:         return Success(value=[{"start_time": 0.0, "end_time": 1.0}])
backend\pipeline\segmenters\motion.py:33:         return Failure(message=f"Could not open
video: {video_path}", is_recoverable=False)
backend\pipeline\segmenters\motion.py:226:         return Success(value=segments_data)

```

4. user (2026-06-10T06:13:04.742Z)

```

50         cap = cv2.VideoCapture(video_path)
51         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
52         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
53         cap.release()
54         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
55
56     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
57                       max_frames: int = None) -> List[Dict[str, Any]]:
58         sport_name = self.config.get("sport", "badminton")
59         try:
60             sport_profile = sport_registry.get(sport_name)
61         except KeyError as e:
62             logger.error(str(e))
63             return []
64
65         idx_cfg = self.config.get("indexing", {})
66         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
67         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
68
69         api_key_env_var = f"{provider_name.upper()}_API_KEY"
70         api_key = os.environ.get(api_key_env_var)
71         if not api_key:
72             logger.error(f"{api_key_env_var} not set; cannot run the {provider_name}
handoff.")
73             return []
74         try:
75             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
76         except KeyError as e:
77             logger.error(str(e))
78             return []
79
80         video_duration = get_video_duration(video_path)
81         if video_duration <= 0:
82             logger.error("Invalid video duration.")
83             return []
84         fps, frame_width = self._probe_fps_width(video_path)
85
86         wasb_cfg = WasbConfig.from_indexing_cfg(idx_cfg)
87         runner: DetectorRunner = WasbRunner(wasb_cfg)
88
89         w = idx_cfg.get("wasb", {})
90         velocity_thresh = w.get("velocity_threshold", 0.02)
91         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
92         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
93         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)

```

```

94     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
95     log_candidates = idx_cfg.get("log_yolo_candidates", True)
96     store_candidates = idx_cfg.get("store_yolo_candidates", True)
97
98     detection_params = {
99         "detector": "wasb_hybrid",
100         "velocity_thresh": velocity_thresh,
101         "merge_gap": merge_gap,
102         "min_action_window_duration": min_window_duration,
103         "weights_path": wasb_cfg.weights_path,
104     }
105
106     ok, msg = runner.healthcheck()
107     if not ok:
108         logger.error("WASB WSL environment not ready: %s", msg)
109         return []
110     logger.info("WASB WSL env OK: %s", msg)
111
112     # --- Phase 2: shuttle trajectory -> candidate rally windows ---
113     t_start = time.time()
114     out_dir = os.path.join("output", "wasb")
115     csv_path = runner.run_predict(video_path, out_dir)
116     if not csv_path:
117         logger.error("WASB produced no trajectory; aborting.")
118         return []
119
120     points = runner.parse_trajectory_csv(csv_path)
121     logger.info("Parsed %d trajectory points (%d visible).",
122                 len(points), sum(1 for p in points if p.visible))
123
124     all_candidate_windows = runner.trajectory_to_action_windows(
125         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
126         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
127         min_window_duration=min_window_duration, chunk_index=0,
128     )
129     logger.info("Phase 2 (WASB) completed in %.1fs. Candidate windows: %d",
130                 time.time() - t_start, len(all_candidate_windows))
131
132     if log_candidates:
133         for cw in all_candidate_windows:
134             logger.info(f"[WASB rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
135                         f"({cw['end'] - cw['start']:.1f}s) | "
136                         f"peak_v={cw['peak_velocity']:.3f} "
137                         f"mean_v={cw['mean_velocity']:.3f}")
138
139     candidate_ids = [None] * len(all_candidate_windows)
140     if store_candidates:

```

5. user (2026-06-10T06:13:05.097Z)

```

1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:

```

```

14         """Unique identifier for this segmenter."""
15         pass
16
17     @property
18     @abstractmethod
19     def description(self) -> str:
20         """Human-readable explanation of this segmenter."""
21         pass
22
23     @abstractmethod
24     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25         """
26         Processes a video to detect play segments, populates SQLite database,
27         and returns a Result.
28
29         Use isinstance(result, Failure) (or result.is_success()) to distinguish
30         "no segments found" from "something went wrong" (missing key, healthcheck
31         failure, etc.). This is the start of the standardized error/result contract.
32         """
33         pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""
41             try:
42                 temp_inst = segmenter_cls(None, {})
43                 name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}")
46             self._segmenters[name] = segmenter_cls
47
48         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49             return self._segmenters.get(name)
50
51         def list_available(self) -> Dict[str, str]:
52             """Returns a dict of registered segmenter names and their descriptions."""
53             res = {}
54             for name, cls in self._segmenters.items():
55                 try:
56                     temp_inst = cls(None, {})
57                     desc = temp_inst.description
58                 except Exception:
59                     desc = "No description available."
60                 res[name] = desc
61             return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

6. user (2026-06-10T06:13:14.910Z)

```

27-from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
28-from backend.pipeline.stitcher.compiler import VideoCompiler
29-from backend.utils.hashing import compute_video_id # canonical file-identity hashing
30-from backend.config import load_config as load_app_config, AppConfig # new typed config
31-from backend.results import Result, Failure # standardized error/result contract
32-from backend.reporting import emit_indexer_report
33-
34-app = FastAPI(title="Sports Highlight Indexer API")
35-

```

```

--
135-         segmenter = segmenter_cls(db_instance, global_config)
136-         result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
137-         segmentation_ran = True
138-
139-         if isinstance(result, Failure):
140-             print(f"[API] Segmenter failed: {result.message}")
141-             response = {
142-                 "video_id": video_id,
143-                 "status": "failed",
--
151-
152-         response = {
153-             "video_id": video_id,
154-             "status": "success",
155-             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
156-             "results": [r.dict() for r in val_results],
157-             "play_segments": segments,
158-         }
159-         return response
--
395-         segmenter = segmenter_cls(db_instance, global_config)
396-         result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
397-         segmentation_ran = True
398-
399-         if isinstance(result, Failure):
400-             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
401-             print(f"[FAIL] Indexing halted for video.")
402-             segments = []
403-         else:

```

7. user (2026-06-10T06:13:16.088Z)

```

backend\pipeline\segmenters\wasb_hybrid.py:63:         return []
backend\pipeline\segmenters\wasb_hybrid.py:73:         return []
backend\pipeline\segmenters\wasb_hybrid.py:78:         return []
backend\pipeline\segmenters\wasb_hybrid.py:83:         return []
backend\pipeline\segmenters\wasb_hybrid.py:109:         return []
backend\pipeline\segmenters\wasb_hybrid.py:118:         return []
backend\pipeline\segmenters\wasb_hybrid.py:152:         return []
backend\pipeline\segmenters\wasb_hybrid.py:166:             return []
backend\pipeline\segmenters\tracknet_hybrid.py:67:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:81:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:86:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:91:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:120:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:131:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:172:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:186:             return []
backend\pipeline\segmenters\gemini.py:36:         return []
backend\pipeline\segmenters\gemini.py:52:         return []
backend\pipeline\segmenters\gemini.py:58:         return []
backend\pipeline\segmenters\gemini.py:263:         return []
backend\pipeline\segmenters\yolo_hybrid.py:127:         return []
backend\pipeline\segmenters\yolo_hybrid.py:163:         return []
backend\pipeline\segmenters\yolo_hybrid.py:230:         return []
backend\pipeline\segmenters\yolo_hybrid.py:272:         return []
backend\pipeline\segmenters\yolo_hybrid.py:288:         return []
backend\pipeline\segmenters\yolo_hybrid.py:294:         return []
backend\pipeline\segmenters\yolo_hybrid.py:301:         return []
backend\pipeline\segmenters\yolo_hybrid.py:494:         return []

```

8. user (2026-06-10T06:13:30.347Z)

```
30- -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend\pipeline\segmenters\base.py::segmenter_registry (400 + available list if missing)
31- -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
32- [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
33- -> finally: emit_indexer_report(...) @backend/reporting/__init__.py (? ALWAYS runs ?
even on validation failure / exception; NEVER raises; grafts response["reports"])
34- -> select segment ids -> [(start,end)] (+ stitching padding)
35- -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
36- per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
--
91-
92-- `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports
`load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`,
`ValidationConfig` (from `.models`); `__all__` = exactly those 6. Use `from backend.config
import load_config, AppConfig`.
93-- `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for defaults**.
94- - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`,
`ai_model='gemini-2.5-flash'`; sections `validation`, `indexing`, `output`, `stitching`
(`default_factory`). `model_config` = {extra:'allow', validate_assignment:True}. ?
`extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
95- - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain dicts**
(`model_dump(mode='json')`, NOT the model) so chained `config.get("indexing",{}).get(...)` keeps
working; falls back to searching a leaf key in any section (sections discovered dynamically from
`model_fields`). ? "bit every validator on real runs" per docstring ? returning the model breaks
`.get`. `::AppConfig.__getitem__` raises `KeyError` for unknown top-level keys else delegates to
`.get`.
96-[Omitted long context line]
97-[Omitted long context line]
--
110-[Omitted long context line]
111-- `@backend\pipeline\segmenters\tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`tracknet_hybrid`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
112-[Omitted long context line]
113-- `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `gemini`) ? pure-AI,
no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
114-- `@backend\pipeline\segmenters\motion.py::MotionPlaySegmenter` (name `motion`) ? pixel
absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
115-
116-### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
--
118-- `@backend/pipeline/detectors/tracknet_runner.py`
119- - `::TrackNetConfig`(+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro`
field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
120- - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/~` passthrough, UNC unhandled),
`::TrajectoryPoint`(frame,visible,x,y).
121-[Omitted long matching line]
122-[Omitted long context line]
123-[Omitted long context line]
124-[Omitted long context line]
125-
126-### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
127-- `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`,
```

```

`::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape),
`::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN)
for mean_iou/boundary errors.
128-- `@backend\eval\annotations.py` ? generic GT loader. `::load_annotations` (`start,end` CSV;
RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
129-- `@backend\eval\calibration.py` ? overfit guard. `::cross_validate` (within-video k-fold; ?
defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video; needs ?2
labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT), `::select_best`,
`::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. $ `PredictFn =
Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap >~0.1` = overfit
alarm.
130-- `@backend\eval\calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` ($`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
131-- `@backend\eval\calibrate_local.py` ? distill Gemini->local thresholds (windowing + net-
crossing gate, no cloud at runtime). `::local_grid` (180 cfgs), `::make_local_predict_fn` (adds
`rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a labels
file yields no rallies), `::run`/`::main`. $
`LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.
132-- `@backend\eval\export_wasb_segments.py` ? tuned cfg -> golden\slicer CSV + comparison +
optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video), `::SEG_FIELDS`, `::COMP_FIELDS`,
`::build_comparison`, `::seconds_to_mms`, `::maybe_slice` (lazy-imports rally_slicer).
`--slice` requires `--video`. ? `write_segments_csv` output loads straight into
`rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is load-bearing.
133-- `@backend\eval\batch.py` ? corpus aggregation (pure; orchestration is in run_phase3_eval).
`::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1), `::default_stages_for`
(`auto`; `::FINAL_ONLY_SEGMENTERS={motion,gemini}`), `::resolve_video_path` (normalizes
collector `./data\..` Windows paths), `::format_aggregate`.
--
139-[Omitted long context line]
140-[Omitted long context line]
141-[Omitted long context line]
142:[Omitted long matching line]
143-
144-### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
145-[Omitted long context line]
--
147-
148-### 2.4 Stitcher / tools / utils
149-[Omitted long context line]
150-- `@backend\tools\rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure, unit-
tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::ClipWindow` (rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (?
duplicate compiler's by copy). `read_time` -> backend\eval\annotations.parse_timestamp`.
151-- `@backend\utils\video.py` ? `::get_video_duration` (ffprobe; ? 0.0 on failure =
"unknown"), `::extract_video_chunk`(...,reencode=False,scale_width=None). ? copy mode cuts at
nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale_width` requires
`reencode=True`.
152-- `@backend\utils\geometry.py` ? `::get_velocity_normalised` (fraction of frame-width/s; ?
raw-px fallback if width<=0; used by yolo_hybrid), `::calculate_iou` (? +1 px convention
inflates IoU), `::get_center/get_distance/get_velocity`.
153-
154-### 2.5 Validators ?
155:[Omitted long matching line]
156-- `@backend\pipeline\validators\__init__.py` ? registers in EXEC order: FormatValidator,
ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator,
RelevanceValidator (? producers before consumers ? format_check first; ? import mutates global
registry as side effect).
157-- `@backend\pipeline\validators\format.py::FormatValidator` (`format_check`) ? **PRODUCER**
of `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is substring

```



```

`mp4` (so MOV-family passes); external `ffprobe`.
158-- `@backend\pipeline\validators\resolution_fps.py::ResolutionFPSValidator`
(`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails "Run format_check first" if
absent). `min_fps` 29.9; min res 1280x720.
--
163-
164-### 2.6 Sports / Providers / Annotations ?
165-[Omitted long context line]
166:[Omitted long matching line]
167-[Omitted long context line]
168-- `@backend\annotations\file_provider.py::FileProvider` (`file`; job_id IS the resolved
CSV path; `_resolve` -> `

```

```

220-- `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet
normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read from
cfg["model"] at runtime in wasb_infer.run.)
221-- `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results,
hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.
(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
222-- `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes every
frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()` resets.
`det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls
`build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y`
? consistent.)
223-- `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh, transform,
seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]}]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)
224-- Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-data-
trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial, avoided.
--
236-- **MD5 keying:** `video_id` = MD5 of WHOLE file via the single
`@backend/utils/hashing.py::compute_video_id` helper (64KB chunks), imported by main + all
run_*.py. Manifest `video_id` is a label, NOT the DB key.
237-- **Config access:** `backend/main.py` and all four `run_*.py` load via
`backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains
for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in
`main.py` that only fires if `global_config`/.indexing` is `None` (the ASGI-import edge case).
238-- **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `motion`
getattr fallback noted above.
239-- **`AppConfig.get` must return dict sections, not models:** the legacy
`.get("indexing",{})` chains everywhere depend on it; the `extra="allow"` model also
silently keeps typo'd config keys (no rejection).
240-- **`output_dir`:** a real field on `OutputConfig` (default `"output"`); the CLI `--output-
dir` overrides it on the typed model in `main()`, and `compile_highlights` reads the same
`global_config.output.output_dir` ? honored end-to-end.
241-- **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if
`{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op. `analyze_video` also
returns `[]` on real failure -> empty != "no rallies". The reporting
`silent_provider_noop`/`ai_empty_vs_no_rallies` rules exist precisely to disambiguate this in
the report.
242-- **Reporting never breaks a run:** `emit_indexer_report` swallows all exceptions (designed
for `main.py` `finally:`) and is emitted even on validation failure. `obsreport` is a SOFT dep ?
absent -> no-op.
243-- **spec.yaml <-> harvest.py coupling:** rule `path` strings dotted-navigate harvest's
stage/metric/detail names; rename a detail in harvest.py and the matching rule silently stops
firing. Add rules in spec.yaml, not code.
244-- **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner fallbacks
(30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes `59.94/1920`.
The report's `fps_fallback_used` flag surfaces this.
245-- **WSL `/mnt` can't see Google Drive / network mounts;** stage video into native WSL fs
(`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
246-- **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on PATH);
WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg +
`--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo
modules).
247-- **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-
reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
--
321-| `test_config.py` | `config` (load_config defaults, save_default_config, AppConfig defaults
pinned, `extra` tolerance, `.get()` dict-compat) |
322-| `test_reporting_harvest.py` | `reporting` (`emit_indexer_report`, `harvest.record_run`); ?
`importorskip("obsreport")`; all 7 footgun rules + plain-dict-OR-AppConfig +
`report.enabled=False` |
323-| `test_reporting_spec_faq.py` | `reporting/spec.yaml` <-> `README.md` cross-ref integrity
(every `faq_ref` resolves; actionable rules cite a `faq_ref`); ? `importorskip("obsreport")` |

```

```

324-| `test_validators_base.py` | `pipeline/validators/base.ValidationRegistry`
(register/run_all/`run_all_with_context` -> `(results,context)`; exception captured not raised)
|
325-| `test_sport_profiles.py` | `sports` registry + BadmintonSportProfile (prompt contains
"decimal seconds"/chunk value; relevance config keys) |
326-| `test_providers.py` / `test_provider_gemini.py` | `providers` registry + GeminiProvider
lifecycle (`MAX_UPLOAD_MB` guard, non-list JSON -> [], cleanup delete) |
327-| `test_annotation_providers.py` / `test_automated_provider.py` | `annotations` registry,
FileProvider, AutomatedProvider (oracle/lineage); keystone one-scorer contract ->
`eval.metrics.score` |

```

9. user (2026-06-10T06:13:31.287Z)

```

5-only after this doc is approved.
6-
7-Why these three: they were cataloged as out-of-scope during the consistency
8-pass because each is larger/design-heavy or needs the owner's machine. They are the
9:remaining high-value hygiene items not already done (config/structure/Result/logging/
10-reports) or in flight (detector abstraction, PR #49).
11-
12-Recommended sequence: #7 Packaging (lowest risk, unblocks clean installs/CI) ?
13-#13 Train/eval guardrails (correctness of the data flywheel) ? #16 Async jobs
--
58-Separation is by convention only. `backend/eval/calibrate_local.py` and
59-`calibrate_wasb.py` do honest leave-one-video-out (LOVO), but nothing enforces
60-that a video used to train the learned segmenter
61-(`backend\eval\training.py::build_training_set` ? `distill_local`) is absent from the
62:eval/regression set. Leakage would silently inflate the very metrics the product
63-story rests on (? in BACKLOG).
64-
65-Approach
66-- Split unit = match, never clip (per `GOLDEN_SET_PHASE1_VIDEOS.md`: split by
--
94-request for the full duration of a long video. Unworkable as a hosted/product API.
95-
96-Approach (minimal first slice ? single self-hosted box)
97-- `POST /api/process` returns 202 + `job_id`; add `GET /api/jobs/{job_id}`
98: returning `{status, progress, result, reports}`.
99-- Run the pipeline on a background worker ? start with a `ThreadPoolExecutor`
100- (segmenters already use threads internally for AI handoff, so this fits); leave a
101- real queue (RQ/Celery) for later if multi-process is needed.
102-- Persist job state in the existing SQLite DB via the migration ladder

```

10. user (2026-06-10T06:13:54.409Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\BACKLOG.md-77-- ? **Per-video observability reports shipped in BOTH tools** (a **TECHNICAL** report + a **CUSTOMER-friendly** summary), via the **soft-dependency** lib `obsreport` (github.com/avidullu/sports-obsreport, MIT, v0.1.3). Each run writes `

docs\BACKLOG.md-78-- ? **Item 8 (Input-Quality UX) is LARGELY SUBSUMED** by these reports (validation stage + footgun flags + verdict already surface quality; a clean customer summary already exists). **Do NOT build a parallel mechanism** ? instead **EXTEND** the report: add an audio-readiness stage/flag to `backend/reporting/` (harvest) + the obsreport spec. **Revisit:** when item 8 is picked up.

docs\BACKLOG.md-79-- ? **Item 7 (unify hybrid segmenters) should land before the report's real AI-provider timing.** The report records AI timing as `not\_captured`; once `wasb\_hybrid`/`tracknet\_hybrid` are unified there is **ONE** place to thread a recorder callback through `provider.analyze\_video` ? the report. **Revisit:** with item 7 (Claude to add the timing hook there).

docs\BACKLOG.md:80-- ? **Item 3 (Error/Result contract) synergizes with the report.** Today the report **INFERS** failures (e.g. `silent\_provider\_noop` = 0 segments + no API key). A structured Result/error from providers/segmenters would let the report read the real reason as a first-class flag. **Revisit:** with item 3.

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\BACKLOG.md-81-- ? **`obsreport` is a**

SOFT/commented dep** in `requirements.txt` (works locally via editable install; pipeline no-ops if absent). **Pin the `git+ssh@v0.1.3` tag** in both repos once SSH/CI access to the private lib repo exists.

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\BACKLOG.md-82-- ?

`run\_all\_tests.py` added at repo root ? run the full suite (clear PASS/FAIL, pytest exit code) before sending PRs.

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\BACKLOG.md-83-- ? **Config-migration follow-ups (DONE ? config & identity consistency pass)** ? the two typed-config bugs from --

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md-194-

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md-195-- **Item 8 (Input-Quality UX) ? mostly already delivered.** The reports already surface validator results + footgun flags + a verdict (technical) *and* a clean customer summary. Reframe item 8 as **extend the report** (e.g. an audio-readiness signal in `backend/reporting/` + the obsreport spec), not a new surface.

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md-196-- **Item 7 (unify hybrid segmenters) ? do this before wiring real AI-provider timing.** The report records AI timing as `not\_captured`; a unified segmenter gives one place to thread the timing in. (Claude will add that hook with/after item 7.)

docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:197:- **Item 3 (Error/Result contract) ? synergy.** A structured Result from providers/segmenters lets the report read failure reasons directly instead of inferring them (e.g. `silent\_provider\_noop`).

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md-198-- **Items 4 (logging), 5 (packaging), 6 (detector runner abstraction) ? safe to do independently** (near-zero overlap with the reports). For #5, model `pyproject.toml` on `obsreport`'s and pin the `obsreport` dep tag.

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md-199-

docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md-200-Also added for the handoff: **`run\_all\_tests.py`** at the repo root ? one-command full suite (clear PASS/FAIL + pytest exit code) before opening PRs.

11. user (2026-06-10T06:14:11.377Z)

178-

179-User merged #50 and #51. Scaffolding phase complete; shift focus to quality improvement with new labeled videos (expected mid-next week). See refined priorities in COMMERCIAL_READINESS_REVIEW.md.

180-

181----

182-

183-## 7. Status & observability-feature intersections (2026-06-07, refreshed)

184-

185-**Done (merged):**

186:- Low-regret 1-6 (Config, structure, Result contract, logging, packaging, Detector Runner ABC finish ? #49)

187-- Scaffolding maturation batch:

188- - 2. Input Quality UX via reports extension (audio readiness signal + capture guidance surfaced in reports from VIDEO_RECORDING_GUIDELINES) ? #50

189- - 3. Value Layer (minimal first slice: EDL/JSON/CSV timeline exports + basic stats over play segments) ? #51

190-

191-All three recommended scaffolding items completed as separate PRs with full tests/coverage/docs discipline. Scaffolding now matured for iteration.

192-

193-In parallel, **per-video observability reports** shipped across both tools (indexer PR #41\#42\#43, collector PR #11\#12) on a new shared, permissively-licensed lib **`obsreport`** (a small "report config language"). Coordination so the remaining items don't collide or duplicate:

194-

195-- **Item 8 (Input-Quality UX) ? mostly already delivered.** The reports already surface validator results + footgun flags + a verdict (technical) *and* a clean customer summary. Reframe item 8 as **extend the report** (e.g. an audio-readiness signal in `backend/reporting/` + the obsreport spec), not a new surface.

196-- **Item 7 (unify hybrid segmenters) ? do this before wiring real AI-provider timing.** The

report records AI timing as `not\_captured`; a unified segmenter gives one place to thread the timing in. (Claude will add that hook with/after item 7.)

197:- ****Item 3 (Error/Result contract) ? synergy.**** A structured Result from providers/segmenters lets the report read failure reasons directly instead of inferring them (e.g. `silent\_provider\_noop`).

198-- ****Items 4 (logging), 5 (packaging), 6 (detector runner abstraction) ? safe to do independently**** (near-zero overlap with the reports). For #5, model `pyproject.toml` on `obsreport`'s and pin the `obsreport` dep tag.

199-

200-Also added for the handoff: ****`run\_all\_tests.py`**** at the repo root ? one-command full suite (clear PASS/FAIL + pytest exit code) before opening PRs.

201-

202-Found while validating the restructure end-to-end: the observability report ****surfaced a real config runtime bug**** (the typed `AppConfig` broke validators' nested `.get`), fixed in PR #42 ? a good signal that the reporting layer earns its keep.

12. user (2026-06-10T06:14:20.462Z)

40-9. (Stretch) First slice of async job model.

41-

42-****Important sequencing decision (approved):**** Complete the Config work ****first**** (this introduces `backend/config/` as a package immediately). Then perform the broader directory refactor as a ****separate, atomic, high-priority PR****.

43-

44-Rationale:

45-- Keeps the Config PR small and focused.

46-- Allows the structural change to land atomically with minimal merge conflicts.

47-- Gives Claude a clean window with no concurrent file moves.

48-- Still delivers the "level of indirection" benefit early via the `config/` package.

49-

50----

51-

52:## 3. Approved Directory Structure Direction

53-

54-Current flat layout under `backend/` (many sibling dirs + root .py files) feels unprofessional for long-term maintenance. We will trend toward clear layers without creating 15+ tiny single-file directories.

55-

56-****Target layered structure (directional, not all at once):****

57-

58-```

59-backend/

60-??? __init__.py

61-??? main.py # Thin CLI + FastAPI bootstrap only

62-?

63-??? config/ # ? Start here with Deliverable 1

64-? ??? __init__.py

13. user (2026-06-10T06:14:28.670Z)

28-

29-## 2. Overall Sequence (Approved Order)

30-

31-1. ****Config Model (Pydantic) + Central Loader**** (as a proper package) ? foundational.

32-2. ****Directory Structure Refactor**** (high-pri, atomic follow-up PR) ? move to the approved layered layout.

33:3. Error / Result Contract standardization.

34-4. Logging standardization + removal of stray `print()` in hot paths. ? (done in feat/logging-standardization; central setup in main; replaced in motion.py, wasb_infer.py, stitcher/compiler.py, config/loader.py, main internal diagnostics; user-facing output in run_*.py and main CLI kept as print where appropriate for direct console UX. Docs updated in CODE_MAP and this plan.)

35-5. Packaging modernization (`pyproject.toml` + clean entry points). ? (done in feat/packaging-modernization; pyproject.toml with PEP 621 + [project.scripts]; pip install -e . supported; setup.py retained for diagnostics; docs + tests updated.)

36-6. Detector Runner Abstraction (start + finish de-risking WSL). ? (start sketched; finished in feat\finish-detector-runner-abstraction + PR #49: base.py ABC now matches the real (ok, msg) contract; runners inherit; hybrids type against DetectorRunner; contract tests for both; rich docs for adding Linux-native runners. 224 passed @64%.)

37-7. Unify / extract shared logic from the two hybrid segmenters.

38-8. Input Quality UX ? reframed as extend existing reports (audio readiness + capture guidance from VIDEO_RECORDING_GUIDELINES). ? (feat/extend-reports-input-quality + PR #50, merged)

--

157-

158----

159-

160-## 6. Later Low-Regret Items (after the above two)

161-

162:- Error / Result Contract standardization (small, high signal). ?

163-- Logging standardization. ?

164-- Packaging modernization. ?

165-- Detector Runner Abstraction (WSL de-risk). ? (feat/finish-detector-runner-abstraction)

166-- Unify hybrid segmenter logic.

167-- Input Quality UX surfacing.

--

181----

182-

183-## 7. Status & observability-feature intersections (2026-06-07, refreshed)

184-

185-**Done (merged):**

186:- Low-regret 1-6 (Config, structure, Result contract, logging, packaging, Detector Runner ABC finish ? #49)

187-- Scaffolding maturation batch:

188- - 2. Input Quality UX via reports extension (audio readiness signal + capture guidance surfaced in reports from VIDEO_RECORDING_GUIDELINES) ? #50

189- - 3. Value Layer (minimal first slice: EDL/JSON/CSV timeline exports + basic stats over play segments) ? #51

190-

191-All three recommended scaffolding items completed as separate PRs with full tests/coverage/docs discipline. Scaffolding now matured for iteration.

192-

193-In parallel, **per-video observability reports** shipped across both tools (indexer PR #41\#42\#43, collector PR #11\#12) on a new shared, permissively-licensed lib **`obsreport`** (a small "report config language"). Coordination so the remaining items don't collide or duplicate:

194-

195-- **Item 8 (Input-Quality UX) ? mostly already delivered.** The reports already surface validator results + footgun flags + a verdict (technical) *and* a clean customer summary. Reframe item 8 as **extend the report** (e.g. an audio-readiness signal in `backend/reporting/` + the obsreport spec), not a new surface.

196-- **Item 7 (unify hybrid segmenters) ? do this before wiring real AI-provider timing.** The report records AI timing as `not\_captured`; a unified segmenter gives one place to thread the timing in. (Claude will add that hook with/after item 7.)

197:- **Item 3 (Error/Result contract) ? synergy.** A structured Result from providers/segmenters lets the report read failure reasons directly instead of inferring them (e.g. `silent\_provider\_noop`).

198-- **Items 4 (logging), 5 (packaging), 6 (detector runner abstraction) ? safe to do independently** (near-zero overlap with the reports). For #5, model `pyproject.toml` on `obsreport`'s and pin the `obsreport` dep tag.

199-

200-Also added for the handoff: **`run\_all\_tests.py`** at the repo root ? one-command full suite (clear PASS/FAIL + pytest exit code) before opening PRs.

201-

202-Found while validating the restructure end-to-end: the observability report **surfaced a real config runtime bug** (the typed `AppConfig` broke validators' nested `.get`), fixed in PR #42 ? a good signal that the reporting layer earns its keep.

14. user (2026-06-10T06:15:32.086Z)

Structured output provided successfully

15. assistant (2026-06-10T06:15:36.189Z)

Verification complete.